INGENIERÍA DE SOFTWARE

Unidad 05: Diseño de Software Conceptual

Conceptos fundamentales · Descomposición y modularidad · Estilos arquitectónicos
Características de un buen diseño · Evaluación y validación · Diseño de la interfaz

Bibliografía base: Pressman, Ingeniería del Software (6.ª ed.) · Sommerville, Ingeniería de Software

Contenido de la clase (1/2)

01 El diseño en la ingeniería de software

02 Conceptos fundamentales de diseño

03 Descomposición y modularidad

04 Cohesión y acoplamiento

05 Arquitectura y estilos arquitectónicos clásicos

06 Estilos actuales y estrategia de selección

Contenido de la clase (2/2)

07	Características de un buen diseño
08	Principios SOLID
09	Evaluación y validación del diseño
10	Diseño de la interfaz de usuario
11	Usabilidad, accesibilidad y prototipos

TEMA 1

El diseño en la ingeniería de software

Qué es diseñar, dónde se ubica en el proceso y qué produce

¿Qué es el diseño de software?

El diseño es el lugar donde los requerimientos se transforman en una representación del software cuya calidad se puede evaluar antes de escribir código.

Diseño conceptual

Define qué componentes tendrá el sistema, qué responsabilidad cumple cada uno y cómo se relacionan. Usa el lenguaje del problema.

Diseño técnico

Precisa cómo se construirá cada componente: algoritmos, estructuras de datos, interfaces concretas y tecnología.

Por qué importa

Es la última oportunidad barata de encontrar errores: un defecto de diseño detectado en el código o en producción cuesta mucho más.

Sin diseño, la estructura del sistema "emerge" del código: funciona… hasta que hay que cambiarlo.

Fuente: Pressman, Ingeniería del Software, Cap. 9 — Ingeniería del diseño.

Del modelo de requerimientos al modelo de diseño

Cada parte del modelo de requerimientos (Unidad 04) alimenta uno o más de los cuatro elementos del modelo de diseño.

Diseño de datos	Transforma las clases y entidades del análisis en estructuras de datos y, cuando corresponde, en el diseño de la base de datos.
Diseño arquitectónico	Define los grandes bloques del sistema, cómo se comunican y qué estilo arquitectónico los organiza.
Diseño de interfaces	Describe cómo se comunica el software con los usuarios, con otros sistemas y entre sus propios componentes.
Diseño de componentes	Detalla la estructura interna y el comportamiento de cada componente: es el nivel más cercano al código.

Fuente: Pressman, Cap. 9 — Ingeniería del diseño (el modelo de diseño).

TEMA 2

Conceptos fundamentales de diseño

Las ideas que todo diseñador usa, sin importar el método ni la tecnología

Seis conceptos de diseño

Abstracción

Concentrarse en lo esencial e ignorar los detalles que no importan en ese nivel. Puede ser procedimental (una acción con nombre) o de datos (un objeto con nombre).

Refinamiento progresivo

Partir de una descripción general y agregar detalle en pasos sucesivos, de arriba hacia abajo (Wirth).

Ocultamiento de información

Cada módulo esconde sus decisiones internas y expone solo lo necesario a través de su interfaz (Parnas).

Separación de intereses

Dividir un problema complejo en partes que puedan resolverse y modificarse de forma independiente.

Patrones

Soluciones probadas a problemas de diseño que se repiten en un contexto dado; dan un vocabulario común al equipo.

Refactorización

Mejorar la estructura interna del diseño o del código sin cambiar su comportamiento externo (Fowler).

Fuente: Pressman, Cap. 9 — Ingeniería del diseño (conceptos de diseño).

Refinamiento progresivo en un ejemplo

La misma tarea vista en niveles crecientes de detalle: cada paso puede volver a refinarse hasta llegar a algo implementable.

Inscribir alumno en una materia

Verificar correlativas › **Verificar cupo** › **Registrar inscripción** › **Emitir comprobante**

Obtener historial del alumno | **Consultar plan de estudios** | **Comparar materias aprobadas**

Nivel 1 → nivel 2 → nivel 3 (refinamiento de "Verificar correlativas").

TEMA 3

Descomposición y modularidad

Cómo dividir un sistema en partes manejables

Modularidad: ni muy pocos ni demasiados

Un sistema modular está dividido en componentes con nombre propio (módulos) que se integran para satisfacer los requerimientos.

Muy pocos módulos

Cada módulo es grande y complejo: cuesta entenderlo, probarlo y modificarlo sin romper algo.

Síntoma: un cambio pequeño obliga a revisar archivos enormes.

Demasiados módulos

Cada módulo es simple, pero crece el esfuerzo de integrarlos y de coordinar la comunicación entre ellos.

Síntoma: para seguir una funcionalidad hay que saltar por decenas de piezas.

Fuente: Pressman, Cap. 9 — Ingeniería del diseño (modularidad). Existe un punto intermedio de costo mínimo.

Estrategias de descomposición

Funcional (descendente)

Se parte de la función principal y se divide en subfunciones. Natural en sistemas orientados a procesos.

Orientada a objetos

Se identifican objetos del dominio con datos y comportamiento; el sistema es una colaboración entre ellos.

Por capas

Se agrupan responsabilidades por nivel de abstracción: presentación, lógica de negocio y acceso a datos.

Por dominio

Se divide según áreas del negocio (ventas, inventario, facturación). Es la base del diseño guiado por el dominio (DDD) y de los microservicios.

Fuente: Pressman, Caps. 9 y 10 · Sommerville, Cap. 6 — Diseño arquitectónico. Las estrategias pueden combinarse.

TEMA 4

Cohesión y acoplamiento

Los dos criterios clave para juzgar si una descomposición es buena

Independencia funcional

Cohesión

Mide qué tan relacionadas están las responsabilidades dentro de un mismo módulo. Un módulo cohesivo hace una sola cosa y la hace bien.

Objetivo: ALTA cohesión.
Mira hacia adentro del módulo.

Acoplamiento

Mide cuánto depende un módulo de otros. Con bajo acoplamiento, un cambio en un módulo casi no afecta al resto.

Objetivo: BAJO acoplamiento.
Mira hacia afuera del módulo.

Fuente: Pressman, Cap. 9 (independencia funcional) y Cap. 11 — Diseño en el nivel de componentes.

Tipos de cohesión: de la peor a la mejor

Coinci-dental › Lógica › Tempo-ral › Procedi-mental › Comuni-cacional › Secuen-cial › **Funcional**

Coincidental	Tareas sin relación, agrupadas por casualidad. Ej.: un módulo "Utilidades" que formatea fechas, envía correos y calcula impuestos.
Temporal	Tareas agrupadas solo porque ocurren al mismo tiempo. Ej.: "Inicializar sistema" (abre la base, carga configuración, limpia temporales).
Funcional	Todas las partes contribuyen a una única tarea bien definida. Ej.: "Calcular promedio del alumno".

Fuente: Pressman, Cap. 11 — Diseño en el nivel de componentes (clasificación de Yourdon y Constantine).

Tipos de acoplamiento: del mejor al peor

De datos › **Por estampado** › **De control** › **Externo** › **Común** › **De contenido**

De datos	Se pasan solo los datos necesarios, como parámetros simples.
De control	Un módulo le pasa a otro una bandera que decide qué hace internamente el que la recibe.
Común	Varios módulos leen y escriben las mismas variables globales: un cambio afecta a todos.
De contenido	Un módulo modifica o depende de los datos internos de otro: viola el ocultamiento de información.

Fuente: Pressman, Cap. 11 — Diseño en el nivel de componentes.

TEMA 5

Arquitectura y estilos arquitectónicos clásicos

La estructura de alto nivel del sistema y las formas probadas de organizarla

¿Qué es la arquitectura de software?

Es la estructura del sistema: sus componentes, las propiedades visibles de esos componentes y las relaciones entre ellos (Bass, Clements y Kazman).

Comunica

Es una representación que permite a todos los interesados discutir el sistema con un lenguaje común.

Decide temprano

Concentra las primeras decisiones de diseño: las de mayor impacto y las más caras de revertir.

Habilita la calidad

Rendimiento, seguridad, mantenibilidad y escalabilidad dependen más de la arquitectura que del código.

Fuente: Pressman, Cap. 10 — Creación de un diseño arquitectónico.

Estilos arquitectónicos clásicos (1/2)

En capas

Cada capa ofrece servicios a la capa superior y usa los de la inferior (presentación → negocio → datos).

Ventaja: los cambios quedan localizados en una capa.

Cliente-servidor

Los servidores ofrecen servicios (datos, impresión, cálculo) y los clientes los consumen a través de una red.

Ventaja: los recursos se centralizan y se comparten.

Centrado en datos

Un repositorio central al que acceden los demás componentes para leer, agregar o modificar información.

Ventaja: los componentes no necesitan conocerse entre sí.

Fuente: Pressman, Cap. 10 (estilos arquitectónicos) · Sommerville, Cap. 6 (patrones arquitectónicos).

Estilos arquitectónicos clásicos (2/2)

Flujo de datos

Tuberías y filtros: los datos atraviesan una serie de componentes que los transforman uno tras otro.

Ejemplo: un compilador o un proceso por lotes.

Llamada y retorno

Un programa principal invoca subprogramas que a su vez invocan a otros: estructura jerárquica.

Ejemplo: muchos sistemas tradicionales.

Modelo-Vista-Controlador

Separa los datos (modelo), su presentación (vista) y el manejo de la interacción (controlador).

Ejemplo: aplicaciones web con ASP.NET MVC o similares.

Fuente: Pressman, Cap. 10 · Sommerville, Cap. 6.

Un ejemplo: sistema de inscripciones en capas



Reglas del estilo

- Cada capa solo usa la capa inmediatamente inferior.
- La presentación nunca accede directo a la base de datos.
- Cambiar el motor de base de datos afecta solo al acceso a datos.
- Agregar una app móvil reutiliza la lógica de negocio existente.

TEMA 6

Estilos actuales y estrategia de selección

Cómo se organizan muchos sistemas modernos y cómo elegir un estilo

Estilos arquitectónicos actuales

Monolito modular

Una sola aplicación que se despliega como una unidad, pero dividida internamente en módulos con límites claros.

Microservicios

Servicios pequeños e independientes, cada uno con sus propios datos, que se despliegan y escalan por separado.

Orientado a eventos

Los componentes publican eventos ("pedido creado") y otros reaccionan a ellos a través de un intermediario, sin conocerse entre sí.

No existe un estilo "moderno" mejor en sí mismo: los microservicios agregan complejidad operativa (red, monitoreo, consistencia de datos) que solo se justifica a cierta escala.

Fuentes: Richards y Ford, Fundamentals of Software Architecture · Newman, Building Microservices · Fowler, "MonolithFirst".

Monolito modular frente a microservicios

Aspecto	Monolito modular	Microservicios
Despliegue	Una sola unidad: simple de publicar	Cada servicio por separado: requiere automatización
Escalado	Se escala la aplicación completa	Se escala solo el servicio que lo necesita
Datos	Una base de datos, transacciones simples	Una base por servicio; la consistencia es más difícil
Complejidad operativa	Baja	Alta: red, monitoreo, trazas, fallos parciales
Cuándo conviene	Equipos chicos, dominio aún cambiante	Equipos grandes y autónomos, partes con cargas muy distintas

Fuentes: Newman, Building Microservices · Richards y Ford, Fundamentals of Software Architecture.

Estrategia de selección: los atributos de calidad mandan

El estilo se elige según los atributos de calidad más importantes para el sistema (los requerimientos no funcionales), no por moda.

Rendimiento	Minimizar la comunicación entre componentes; agrupar las operaciones críticas.
Mantenibilidad	Capas y módulos con alta cohesión y bajo acoplamiento, fáciles de reemplazar.
Escalabilidad	Componentes sin estado, microservicios o comunicación por eventos.
Seguridad	Arquitectura en capas con controles en los bordes y mínima superficie expuesta.
Disponibilidad	Redundancia y componentes independientes que puedan fallar por separado.

Fuente: Sommerville, Cap. 6 — Diseño arquitectónico (decisiones de diseño arquitectónico).

Vistas de la arquitectura: el modelo 4+1

Vista lógica

Funcionalidad: clases y objetos clave.
Para usuarios y analistas.

Vista de desarrollo

Organización del código en módulos y paquetes.
Para programadores.

+1
Escenarios

Vista de procesos

Procesos en ejecución, concurrencia y comunicación.
Para integradores.

Vista física

Despliegue en servidores, redes y dispositivos.
Para ingenieros de sistemas.

Fuentes: Kruchten, "The 4+1 View Model of Architecture" (1995) · Sommerville, Cap. 6.

TEMA 7

Características de un buen diseño

Cómo reconocer un diseño de calidad y cómo detectar uno malo

Las tres directrices de calidad de Pressman

Un buen diseño debe cumplir, como mínimo, estas tres condiciones:

Implementar los requerimientos

Todos los explícitos del modelo de requerimientos y también los implícitos que el cliente espera.

Ser legible y comprensible

Para quienes van a construirlo, para quienes van a probarlo y para quienes luego lo mantendrán.

Dar una visión completa

De los datos, las funciones y el comportamiento, desde la perspectiva de la implementación.

Fuente: Pressman, Cap. 9 — Ingeniería del diseño (lineamientos de calidad).

Atributos de calidad FURPS

Funcionalidad	Conjunto de capacidades que ofrece, generalidad de las funciones y seguridad del sistema.
Usabilidad	Factores humanos, estética, consistencia y documentación.
Confiabilidad	Frecuencia y gravedad de las fallas, exactitud de los resultados, capacidad de recuperación.
Rendimiento	Velocidad de procesamiento, tiempo de respuesta, consumo de recursos y eficiencia.
Soportabilidad	Facilidad para extender, adaptar, probar, configurar y mantener el sistema.

Fuente: Pressman, Cap. 9 — Ingeniería del diseño (atributos de calidad).

Principios generales de diseño

Alta cohesión, bajo acoplamiento

El principio que resume la independencia funcional: módulos enfocados y poco dependientes entre sí.

KISS — Mantenerlo simple

La solución más simple que resuelve el problema suele ser la mejor. La complejidad tiene que ganarse su lugar.

DRY — No repetirse

Cada regla o dato del sistema debe tener una única representación. Duplicar obliga a cambiar en varios lugares.

YAGNI — No lo vas a necesitar

No diseñar funciones "por las dudas": se agregan cuando un requerimiento real las pide.

Fuentes: Hunt y Thomas, The Pragmatic Programmer (DRY) · Beck, Extreme Programming Explained (YAGNI).

Señales de un mal diseño

Rigidez

Un cambio simple obliga a modificar muchos módulos.

Fragilidad

Un cambio rompe partes que aparentemente no tenían relación.

Inmovilidad

Cuesta reutilizar un componente porque arrastra demasiadas dependencias.

Viscosidad

Es más fácil hacer "un parche" que hacer las cosas bien.

Complejidad innecesaria

Estructuras pensadas para necesidades que nunca llegaron.

Repetición innecesaria

La misma lógica copiada en varios lugares del sistema.

Fuente: Robert C. Martin, Agile Software Development: Principles, Patterns, and Practices (2002).

TEMA 8

Principios SOLID

Cinco principios para diseños orientados a objetos fáciles de mantener

SOLID en una mirada

S · Responsabilidad única	Una clase debe tener una sola razón para cambiar.
O · Abierto / cerrado	Abierto a la extensión, cerrado a la modificación: se agrega comportamiento sin tocar lo que ya funciona.
L · Sustitución de Liskov	Un subtipo debe poder usarse donde se espera su tipo base sin que el programa deje de funcionar.
I · Segregación de interfaces	Mejor varias interfaces pequeñas y específicas que una general que obliga a implementar métodos que no se usan.
D · Inversión de dependencias	Los módulos de alto nivel no dependen de los de bajo nivel: ambos dependen de abstracciones.

Fuentes: Robert C. Martin, Agile Software Development (2002) y Clean Architecture (2017).

Responsabilidad única e inversión de dependencias

Antes

GestorInscripcion valida correlativas, guarda en SQL Server, envía el correo de confirmación y genera el PDF del comprobante.

Cuatro razones para cambiar: cualquier cambio obliga a modificar y volver a probar la misma clase.

Después

ValidadorCorrelativas · IRepositorioInscripciones · INotificador · GeneradorComprobante. GestorInscripcion solo coordina y depende de interfaces.

Se puede cambiar la base de datos, o probar con un repositorio simulado, sin tocar las reglas de negocio.

Fuente: Robert C. Martin, Clean Architecture (2017).

Abierto / cerrado en código (C#)

```
public interface IDescuento {
    decimal Aplicar(decimal monto);
}
public class DescuentoBeca : IDescuento {
    public decimal Aplicar(decimal monto) => monto * 0.5m;
}
public class DescuentoHermanos : IDescuento {
    public decimal Aplicar(decimal monto) => monto * 0.9m;
}
// Nuevo descuento = nueva clase; CalculadoraCuota no se modifica
```

CalculadoraCuota recibe una lista de IDescuento y los aplica: está cerrada a la modificación, pero abierta a nuevos tipos de descuento.

TEMA 9

Evaluación y validación del diseño

Cómo comprobar que el diseño es correcto antes de construirlo

Verificar y validar el diseño

Evaluar un diseño responde dos preguntas: ¿satisface los requerimientos? (validación) y ¿está bien construido? (verificación de su calidad).

Revisiones técnicas

Inspecciones y recorridos (walkthroughs): un grupo examina el diseño buscando errores, omisiones e inconsistencias.

Trazabilidad

Cada requerimiento debe rastrearse hasta los elementos de diseño que lo satisfacen, y cada elemento hasta su requerimiento.

Prototipos

Un prototipo arquitectónico o de interfaz permite probar las decisiones más riesgosas con poco costo.

Fuente: Pressman, Cap. 9 — Ingeniería del diseño y capítulos de gestión de la calidad (revisiones técnicas).

Matriz de trazabilidad: requerimiento → diseño

Req.	Descripción	Elementos de diseño que lo satisfacen
RF-01	Inscribir al alumno en una materia	Módulo Inscripciones · ValidadorCorrelativas · IRepositorioInscripciones
RF-02	Consultar el historial académico	Módulo Académico · Vista Historial
RF-03	Notificar la inscripción por correo	INotificador · NotificadorCorreo
RNF-01	Responder en menos de 2 segundos	Caché de la oferta académica · índices en la base de datos

Una fila sin elementos de diseño revela un requerimiento olvidado; un componente que no aparece en ninguna fila quizás sobra.

Evaluación de arquitecturas: método ATAM

El método de análisis de compromisos arquitectónicos (ATAM) evalúa arquitecturas candidatas frente a los atributos de calidad.

1. **Reunir escenarios** ›
2. **Obtener requisitos y restricciones** ›
3. **Describir estilos candidatos** ›
4. **Evaluar atributos de calidad** ›
5. **Identificar sensibilidades** ›
6. **Criticar y elegir**

Sensibilidad: cuánto cambia un atributo de calidad (por ejemplo, el rendimiento) cuando se modifica una decisión de la arquitectura.

Fuente: Pressman, Cap. 10 — Creación de un diseño arquitectónico (análisis de diseños alternativos).

Métricas sencillas de diseño

Fan-in / fan-out	Cuántos módulos usan a un módulo (fan-in) y cuántos usa él (fan-out). Un fan-out alto indica mucho acoplamiento.
Falta de cohesión	LCOM: mide cuántos métodos de una clase no comparten atributos. Valores altos sugieren dividir la clase.
Complejidad ciclomática	Cantidad de caminos independientes en un componente: más caminos, más difícil de entender y de probar.
Tamaño	Líneas, métodos o responsabilidades por clase. Una clase enorme rara vez tiene una sola responsabilidad.

Fuente: Pressman, capítulo de métricas del producto (métricas del modelo de diseño).

TEMA 10

Diseño de la interfaz de usuario

Cómo diseñar la comunicación entre las personas y el sistema

Tres tipos de interfaz

El diseño de interfaces define cómo fluye la información hacia el sistema, desde el sistema y entre sus propias partes.

Entre componentes

Contratos internos: qué operaciones ofrece cada módulo, qué recibe y qué devuelve.

Con sistemas externos

Integraciones con otros sistemas: servicios web, API, archivos de intercambio, pasarelas de pago.

Con el usuario

Pantallas, formularios, mensajes y navegación: es lo único del sistema que el usuario ve.

Fuente: Pressman, Cap. 9 (elementos del diseño de interfaz) y Cap. 12 — Diseño de la interfaz de usuario.

Las reglas de oro (Theo Mandel)

Dar el control al usuario

Permitir interrumpir, deshacer y elegir el camino.

No obligar a acciones innecesarias.

Ocultar los detalles técnicos internos.

Reducir la carga de memoria

No exigir recordar datos de una pantalla a otra.

Ofrecer valores por defecto útiles.

Mostrar la información de forma progresiva.

Lograr una interfaz consistente

Presentar la información y las acciones siempre de la misma forma.

Respetar las convenciones que el usuario ya conoce.

Fuente: Pressman, Cap. 12 — Diseño de la interfaz de usuario (las reglas doradas).

El proceso de diseño de la interfaz

Es un proceso iterativo (en espiral): cada vuelta mejora la interfaz con lo aprendido al validarla con usuarios reales.

Análisis de usuarios, tareas y entorno › **Diseño de la interfaz** › **Construcción del prototipo** › **Validación con usuarios**

Usuarios

Quiénes son, qué experiencia tienen con el dominio y con la tecnología.

Tareas

Qué hacen, en qué orden y con qué frecuencia. Los casos de uso son el punto de partida.

Entorno

Dónde y con qué dispositivo: oficina, celular en la calle, aula con proyector, mala conexión.

Fuente: Pressman, Cap. 12 — Diseño de la interfaz de usuario.

TEMA 11

Usabilidad, accesibilidad y prototipos

Cómo evaluar una interfaz y representarla antes de construirla

Las 10 heurísticas de usabilidad de Nielsen

1. Visibilidad del estado del sistema
2. Relación entre el sistema y el mundo real
3. Control y libertad del usuario
4. Consistencia y estándares
5. Prevención de errores
6. Reconocer antes que recordar
7. Flexibilidad y eficiencia de uso
8. Diseño estético y minimalista
9. Ayudar a reconocer y recuperarse de errores
10. Ayuda y documentación

Fuente: Jakob Nielsen, "10 Usability Heuristics for User Interface Design" (1994), Nielsen Norman Group.

Accesibilidad: diseñar para todas las personas

Contraste suficiente

Texto oscuro sobre fondo claro (o al revés), legible también con poca luz o con un proyector de baja calidad.

No depender solo del color

Un error no se marca únicamente en rojo: se acompaña con un ícono o un mensaje de texto.

Uso completo con teclado

Todas las acciones deben poder hacerse sin ratón, con un orden de navegación lógico.

Etiquetas y textos alternativos

Cada campo con su etiqueta y cada imagen con su descripción, para lectores de pantalla.

Fuente: W3C, Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2.

Del boceto al prototipo

Boceto en papel

Rápido y descartable: sirve para explorar ideas con el usuario en pocos minutos.

Wireframe

Estructura de la pantalla en escala de grises: qué elementos hay y dónde, sin colores ni estilo.

Prototipo interactivo

Simula la navegación (Figma, Penpot o HTML simple) y permite validar tareas con usuarios reales.

Menor costo, más ideas ——————————————→ **Mayor fidelidad, más validación**

Fuente: Pressman, Cap. 12 — Diseño de la interfaz de usuario (prototipos y evaluación del diseño).

Ejemplo de wireframe: inscripción a materias



Indicador de pasos

→ *Visibilidad del estado del sistema*

Cupo visible en cada materia

→ *Reconocer antes que recordar*

Materia sin correlativa bloqueada, con aviso

→ *Prevención de errores*

"Cancelar" siempre disponible

→ *Control y libertad del usuario*

Fuentes bibliográficas

Pressman, Roger S.

Ingeniería del Software: Un Enfoque Práctico (6.ª edición)

Caps. 9 a 12 — Ingeniería del diseño, diseño arquitectónico, diseño en el nivel de componentes y diseño de la interfaz de usuario. Fuente principal de esta unidad.

Sommerville, Ian

Ingeniería de Software

Caps. 6 y 7 — "Diseño arquitectónico" / "Diseño e implementación" (numeración de ediciones recientes; puede variar). Complemento de esta unidad.

Lecturas complementarias

Robert C. Martin	Agile Software Development: Principles, Patterns, and Practices (2002) · Clean Architecture (2017) — SOLID y síntomas de mal diseño.
Richards y Ford	Fundamentals of Software Architecture (2020) — estilos arquitectónicos actuales y atributos de calidad.
Sam Newman	Building Microservices (2.ª ed., 2021) — microservicios y cuándo (no) usarlos.
Philippe Kruchten	"The 4+1 View Model of Architecture", IEEE Software (1995) — vistas de la arquitectura.
Nielsen · W3C	10 heurísticas de usabilidad (nngroup.com) · WCAG 2.2 (w3.org/WAI) — usabilidad y accesibilidad.

En resumen

Un buen diseño se reconoce porque es fácil de entender, de cambiar y de usar.

- Diseñar es descomponer el problema en módulos con alta cohesión y bajo acoplamiento, organizados según un estilo arquitectónico elegido por sus atributos de calidad.
- Principios como KISS, DRY y SOLID ayudan a evitar diseños rígidos y frágiles; revisiones, trazabilidad y ATAM permiten evaluarlos antes de construir.
- La interfaz es la parte del diseño que el usuario ve: las reglas de oro, las heurísticas y los prototipos ayudan a que sea usable y accesible.

Próximo paso: ejercitario de la Unidad 05 y tercera entrega del Trabajo Práctico (Diseño)